Assignment: Big Data Systems

Task: Solve the following for "sf-fire-calls.csv" dataset.

Python file: https://drive.google.com/file/d/1nadAdHka08bKlUblofvzsAcrf3Wog56D/view?usp=sharing

Importing Dataset and Creating SparkSession:

```
filepath = "D:\BDA 20\ABD-BDA-LabWork\datasets\sf-fire-calls.csv"

def create_session():
   spark = SparkSession.builder.appName('Fire Example').getOrCreate()
   return spark

def create_df(spark, filepath):
   df = spark.read.csv(filepath, header = True, inferSchema = True)
   df1 = df.select('CallType', 'CallDate', 'City', 'Zipcode', 'Neighborhood', 'Delay')      #we filter
column that we needed
   return df1

def clean_dataSet(df):
   df1 = df.withColumn('Date', to_date(col('CallDate'), 'MM/dd/yyyy')).drop('CallDate')
   df2 = df1.withColumn('Year',year(col('Date')))\
   .withColumn('Month',month(col('Date')))\
   .withColumn('Week',weekofyear(col('Date')))

   return df2

spark = create_session()
df = create_df(spark, filepath)
df = clean_dataSet(df)
df.printSchema()
```

1. Get yearly count of fire calls

Code:

```
def yearly_count(data):

   return data.groupBy('Year').count().orderBy('Year')

yearly_count(clean).show(30)
```

2. What were all the different types of fire calls in 2018?

Code:

```
def calltypes_in_year(data, yr):
   return data.filter(col('Year') == yr).select('CallType').distinct()

calltypes_in_year(clean, 2018).show(30, truncate=False)
```

3. Which week in the year in 2018 had the most fire calls?
Code:

```
def busiest_week(data, yr):
   return data.filter(col('Year') == yr) \
      .groupBy('Week').count().orderBy(desc('count'))

week_counts_2018 = busiest_week(clean, 2018)
week_counts_2018.show(5)

top_week = week_counts_2018.first()
print(f"Week {top_week['Week']} of 2018 had the most fire calls: {top_week['count']}")
```

4. Get monthly count of fire calls based on year

Code:

```
def monthly_count(data):
   return data.groupBy('Year', 'Month').count().orderBy('Year', 'Month')


monthly_count(clean).show(50)
```

5. Give monthly report of fire call types for selected year
Code:

```
def monthly_calltype_report(data, yr):
   return data.filter(col('Year') == yr) \
      .groupBy('Month', 'CallType').count() \
      .orderBy('Month', desc('count'))

monthly_calltype_report(clean, selected_year).show(100, truncate=False)
```

6. Give top five fire call types for every season of selected year (seasons are like Spring, summer, fall winter etc).

Code:

```
def top5_calltypes_by_season(data, yr):
   season_counts = data.filter(col('Year') == yr) \
      .groupBy('Season', 'CallType').count()

   w = Window.partitionBy('Season').orderBy(desc('count'))
   return season_counts.withColumn('rank', row_number().over(w)) \
      .filter(col('rank') <= 5) \
      .orderBy('Season', 'rank')

top5_calltypes_by_season(clean, selected_year).show(40, truncate=False)
```

7. Whether fire type calls are seasonal?

Code:

```
def calltype_season_share(data):
   season_totals = data.groupBy('Season').count().withColumnRenamed('count', 'season_total')
   calltype_season = data.groupBy('Season', 'CallType').count()

   return calltype_season.join(season_totals, on='Season') \
      .withColumn('pct_of_season', round(col('count') / col('season_total') * 100, 2)) \
      .orderBy('CallType', 'Season')

calltype_season_share(clean).show(50, truncate=False)
```

8. What months within the year 2018 saw the highest number of fire calls?

Code:

```
def monthly_count_in_year(data, yr):
   return data.filter(col('Year') == yr) \
      .groupBy('Month').count().orderBy(desc('count'))


months_2018 = monthly_count_in_year(clean, 2018)
months_2018.show(12)


top_month_2018 = months_2018.first()
print(f"Highest month in 2018: Month {top_month_2018['Month']} ({top_month_2018['count']} calls)")
```

9. Find which type of fire call is major calltype in each year

Code:

```
def major_calltype_per_year(data):
   year_calltype_counts = data.groupBy('Year', 'CallType').count()
   w = Window.partitionBy('Year').orderBy(desc('count'))
   return year_calltype_counts.withColumn('rank', row_number().over(w)) \
      .filter(col('rank') == 1) \
      .drop('rank') \
      .orderBy('Year')

major_calltype_per_year(clean).show(30, truncate=False)
```

10. Find out average delay in response for each call type

Code:

```
def avg_delay_by_calltype(data):
   return data.groupBy('CallType') \
```

```
        .agg(round(avg('Delay'), 2).alias('avg_delay')) \
        .orderBy(desc('avg_delay'))

delay_by_type = avg_delay_by_calltype(clean)
delay_by_type.show(30, truncate=False)
```

11. Find which calltype has maximum average delay time.
Code:

```
max_delay_calltype = delay_by_type.first()
print(f"Call type with maximum average delay: {max_delay_calltype['CallType']} "
    f"({max_delay_calltype['avg_delay']} minutes)")
```

12. Which neighborhood in San Francisco generated the most fire calls in 2018?

Code:

```
def calls_by_neighborhood(data, yr):
   return data.filter(col('Year') == yr) \
      .groupBy('Neighborhood').count().orderBy(desc('count'))
neighborhood_counts_2018 = calls_by_neighborhood(clean, 2018)
neighborhood_counts_2018.show(10, truncate=False)
top_neighborhood_2018 = neighborhood_counts_2018.first()
print(f"Neighborhood with most fire calls in 2018: {top_neighborhood_2018['Neighborhood']} "
    f"({top_neighborhood_2018['count']} calls)")
```

13. Which neighborhoods had the worst response times to fire calls in 2018?

Code:

```
def delay_by_neighborhood(data, yr):
   return data.filter(col('Year') == yr) \
      .groupBy('Neighborhood') \
      .agg(round(avg('Delay'), 2).alias('avg_delay'), count('*').alias('num_calls')) \
      .orderBy(desc('avg_delay'))

delay_by_neighborhood(clean, 2018).show(10, truncate=False)
```

14. Find out calltype whose average response delay time is maximum, increases, decreases or has no relation over years.

Code:

```
def delay_trend_by_calltype(data):
   yearly_delay = data.groupBy('Year', 'CallType') \
      .agg(round(avg('Delay'), 2).alias('avg_delay'))
   return yearly_delay.groupBy('CallType') \
      .agg(round(corr('Year', 'avg_delay'), 3).alias('year_delay_correlation')) \
      .orderBy(desc('year_delay_correlation'))


trend = delay_trend_by_calltype(clean)


trend.show(30, truncate=False)

trend.filter(col('CallType') == max_delay_calltype['CallType']).show(truncate=False)
```

15. For each year find out which city has more calltypes

Code:

```
   year_city_calltypes = data.groupBy('Year', 'City') \
      .agg(count_distinct('CallType').alias('distinct_calltypes'))

   w = Window.partitionBy('Year').orderBy(desc('distinct_calltypes'))
   return year_city_calltypes.withColumn('rank', row_number().over(w)) \
      .filter(col('rank') == 1) \
      .drop('rank') \
      .orderBy('Year')

top_city_by_calltypes(clean).show(30, truncate=False)
```

16. For every year find count of calltypes for 5 cities which has more calls.

Code:

```
def top5_cities_calltype_counts(data):
   year_city_calltypes = data.groupBy('Year', 'City') \
      .agg(count_distinct('CallType').alias('distinct_calltypes'))
   year_city_total_calls = data.groupBy('Year', 'City').agg(count('*').alias('total_calls'))

   w = Window.partitionBy('Year').orderBy(desc('total_calls'))
   top5_cities = year_city_total_calls.withColumn('rank', row_number().over(w)) \
      .filter(col('rank') <= 5) \
      .drop('rank')
   return top5_cities.join(year_city_calltypes, on=['Year', 'City']) \
      .orderBy('Year', desc('total_calls'))

top5_cities_calltype_counts(clean).show(100, truncate=False)
```

17. Is there a correlation between neighborhood, zip code, and number of fire calls?

Code:

```
zip_calls = clean.groupBy("Zipcode") \
     .agg(count("*").alias("fire_calls")) \
     .orderBy(desc("fire_calls")) zip_calls.show(10, truncate=False)

zip_correlation = zip_calls.select( corr("Zipcode", "fire_calls").alias("correlation") )
zip_correlation.show()
```